

finetuning_tutorial

December 10, 2025

1 Homework 5: LLM Fine-tuning with Transformers

In this homework you will be **finetuning a small instruction-tuned Language Model (LLM) to do well on previous 189 exam problems while maintaining its general knowledge capabilities**. We will be evaluating you on a hidden test set containing 189 exam problems and general knowledge questions, all in multiple choice format.

We will walk you through how to fine-tune on custom datasets using the standard Hugging Face **transformers** and **trl** libraries.

In this notebook, we use **Qwen/Qwen2.5-0.5B-Instruct**. This is a small but capable model (0.5 billion parameters) that fits easily on most GPUs and trains quickly, allowing us to perform **full fine-tuning** (updating all weights) rather than needing parameter-efficient methods like LoRA (although you are welcome to use LoRA instead).

In this notebook, we provide a small subset of CS189 Exam Questions for you to test and a walkthrough of a simple finetuning pipeline. The actual test set you would be making predictions will be a mix of different questions (Full details are provided in the accompanying PDF)

1.1 Overview

Your task is to: 1. **Adapt the provided notebook** 2. **Generate predictions** on the private test questions (test.csv) 3. **Submit** your results to Kaggle

Kaggle competition link:

<https://www.kaggle.com/t/11c8ffdc967fe3f27755cde6fb5810e8>

1.2 The Finetuning Pipeline

Now let's walk through a simple finetuning pipeline using the Hugging Face **transformers** and **trl** libraries. Even though we are using a smaller model, the core structure of the finetuning pipeline follows the classic **ML Lifecycle** covered in lecture!

1.3 Learning Problem (P)

Goal: Decide what behavior we want the LLM to learn, and from what data.

We want to fine-tune our base Qwen model to better perform on CS189-style multiple choice questions. Our objective is to minimize cross-entropy loss on a dataset of these questions.

Concretely, we will: - **Load the CS189 MCQ dataset:** A CSV file containing questions, options (A-E), and the correct answer. - **Format the data:** Convert each row into a “chat” format that the model understands. - User: The question + options. - Assistant: The correct answer (e.g., `\boxed{A}`).

1.4 Optimization (M)

Goal: Train the model on the dataset by minimizing loss.

We use TRL’s SFTTrainer (Supervised Fine-Tuning Trainer) to perform gradient-based optimization: - **Loss function:** Standard token-level cross-entropy loss. - **Optimizer:** AdamW (8-bit version to save some memory, though standard AdamW may also fit depending on your GPU). - **Hyperparameters:** Learning rate, batch size, etc.

1.5 Predict & Evaluate (O)

Goal: Check whether the fine-tuned model behaves as desired.

After training, we: - **Run inference:** Ask the model to answer the MCQs. - **Compute accuracy:** Check if the model’s output (parsed from `\boxed{X}`) matches the ground truth. - **Compare:** We measure accuracy *before* and *after* fine-tuning to quantify improvement.

1.6 Part 0: Environment Setup

This cell installs and imports the required libraries.

```
[1]: import sys
IS_COLAB = 'google.colab' in sys.modules
if IS_COLAB:
    from google.colab import drive
    drive.mount('/content/drive')
    %cd /content/drive/MyDrive/cs189/hw/hw5
    ! pip install -q transformers==4.57.2 accelerate datasets trl bitsandbytes
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call `drive.mount("/content/drive", force_remount=True)`.
/content/drive/MyDrive/cs189/hw/hw5

```
[2]: !pip install sentence-transformers faiss-cpu
import os
import re
import math
import pandas as pd
import torch
import faiss
from datasets import Dataset, load_dataset, load_from_disk, Value,
↳ concatenate_datasets
```

```

from trl import SFTTrainer, SFTConfig
from transformers import AutoModelForCausalLM, AutoTokenizer
import matplotlib.pyplot as plt
import random

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
#MAKE SURE YOU ARE USING GPU
print('Using device:', device)

```

```

Requirement already satisfied: sentence-transformers in
/usr/local/lib/python3.12/dist-packages (5.1.2)
Requirement already satisfied: faiss-cpu in /usr/local/lib/python3.12/dist-
packages (1.13.1)
Requirement already satisfied: transformers<5.0.0,>=4.41.0 in
/usr/local/lib/python3.12/dist-packages (from sentence-transformers) (4.57.2)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages
(from sentence-transformers) (4.67.1)
Requirement already satisfied: torch>=1.11.0 in /usr/local/lib/python3.12/dist-
packages (from sentence-transformers) (2.9.0+cu126)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-
packages (from sentence-transformers) (1.6.1)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages
(from sentence-transformers) (1.16.3)
Requirement already satisfied: huggingface-hub>=0.20.0 in
/usr/local/lib/python3.12/dist-packages (from sentence-transformers) (0.36.0)
Requirement already satisfied: Pillow in /usr/local/lib/python3.12/dist-packages
(from sentence-transformers) (11.3.0)
Requirement already satisfied: typing_extensions>=4.5.0 in
/usr/local/lib/python3.12/dist-packages (from sentence-transformers) (4.15.0)
Requirement already satisfied: numpy<3.0,>=1.25.0 in
/usr/local/lib/python3.12/dist-packages (from faiss-cpu) (2.0.2)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-
packages (from faiss-cpu) (25.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-
packages (from huggingface-hub>=0.20.0->sentence-transformers) (3.20.0)
Requirement already satisfied: fsspec>=2023.5.0 in
/usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.20.0->sentence-
transformers) (2025.3.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-
packages (from huggingface-hub>=0.20.0->sentence-transformers) (6.0.3)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-
packages (from huggingface-hub>=0.20.0->sentence-transformers) (2.32.4)
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in
/usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.20.0->sentence-
transformers) (1.2.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-
packages (from torch>=1.11.0->sentence-transformers) (75.2.0)

```

Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformers) (1.14.0)

Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformers) (3.6)

Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformers) (3.1.6)

Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformers) (12.6.77)

Requirement already satisfied: nvidia-cuda-runtime-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformers) (12.6.77)

Requirement already satisfied: nvidia-cuda-cupti-cu12==12.6.80 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformers) (12.6.80)

Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformers) (9.10.2.21)

Requirement already satisfied: nvidia-cublas-cu12==12.6.4.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformers) (12.6.4.1)

Requirement already satisfied: nvidia-cufft-cu12==11.3.0.4 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformers) (11.3.0.4)

Requirement already satisfied: nvidia-curand-cu12==10.3.7.77 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformers) (10.3.7.77)

Requirement already satisfied: nvidia-cusolver-cu12==11.7.1.2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformers) (11.7.1.2)

Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformers) (0.7.1)

Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformers) (2.27.5)

Requirement already satisfied: nvidia-nvshmem-cu12==3.3.20 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformers) (3.3.20)

Requirement already satisfied: nvidia-nvtx-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformers) (12.6.77)

Requirement already satisfied: nvidia-nvjitlink-cu12==12.6.85 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-

```

transformers) (12.6.85)
Requirement already satisfied: nvidia-cufile-cu12==1.11.1.6 in
/usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-
transformers) (1.11.1.6)
Requirement already satisfied: triton==3.5.0 in /usr/local/lib/python3.12/dist-
packages (from torch>=1.11.0->sentence-transformers) (3.5.0)
Requirement already satisfied: regex!=2019.12.17 in
/usr/local/lib/python3.12/dist-packages (from
transformers<5.0.0,>=4.41.0->sentence-transformers) (2025.11.3)
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in
/usr/local/lib/python3.12/dist-packages (from
transformers<5.0.0,>=4.41.0->sentence-transformers) (0.22.1)
Requirement already satisfied: safetensors>=0.4.3 in
/usr/local/lib/python3.12/dist-packages (from
transformers<5.0.0,>=4.41.0->sentence-transformers) (0.7.0)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-
packages (from scikit-learn->sentence-transformers) (1.5.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in
/usr/local/lib/python3.12/dist-packages (from scikit-learn->sentence-
transformers) (3.6.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in
/usr/local/lib/python3.12/dist-packages (from
sympy>=1.13.3->torch>=1.11.0->sentence-transformers) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in
/usr/local/lib/python3.12/dist-packages (from jinja2->torch>=1.11.0->sentence-
transformers) (3.0.3)
Requirement already satisfied: charset_normalizer<4,>=2 in
/usr/local/lib/python3.12/dist-packages (from requests->huggingface-
hub>=0.20.0->sentence-transformers) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-
packages (from requests->huggingface-hub>=0.20.0->sentence-transformers) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.12/dist-packages (from requests->huggingface-
hub>=0.20.0->sentence-transformers) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.12/dist-packages (from requests->huggingface-
hub>=0.20.0->sentence-transformers) (2025.11.12)
Using device: cuda

```

1.7 Part 1: Configuration & Model Loading

Here we define all our settings and load the base model.

Model Design (L): We select Qwen/Qwen2.5-0.5B-Instruct.

```

[3]: 5# =====
# === CONFIGURATION - ALL SETTINGS IN ONE PLACE ===

```

```
# =====

# --- Model Configuration ---
MODEL_NAME = "Qwen/Qwen2.5-0.5B-Instruct" # YOU CANNOT CHANGE THIS

# --- Dataset Configuration ---
#TODO: REPLACE WITH YOUR OWN PATH
MCQ_CSV_PATH = "hw5_sample_eval.csv" # Path to CS189 MCQ sample eval dataset

# --- Training Configuration (feel free to adjust!) ---
TRAIN_BATCH_SIZE = 1
GRADIENT_ACCUMULATION_STEPS = 4
WARMUP_STEPS = 5
MAX_STEPS = 50 # or set num_train_epochs instead
LEARNING_RATE = 1e-5
WEIGHT_DECAY = 0.01
LR_SCHEDULER_TYPE = "linear"
OPTIM = "adamw_8bit" # requires bitsandbytes
SEED = 189

# --- Evaluation Configuration ---
EVAL_MAX_NEW_TOKENS = 64 # How many tokens to generate for inference
OUTPUT_DIR = "./mcq_finetuned_model"
```

```
[4]: # # === Load base model & tokenizer ===
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)

# # Ensure we have a pad token for training
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

model = AutoModelForCausalLM.from_pretrained(
    MODEL_NAME,
    torch_dtype=torch.float16 if torch.cuda.is_available() else torch.float32,
    device_map="auto" if torch.cuda.is_available() else None,
)

model.resize_token_embeddings(len(tokenizer))
model.to(device)
model.eval()
print('Model loaded.')
```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94:
UserWarning:
The secret `HF\_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab
(<https://huggingface.co/settings/tokens>), set it as secret in your Google Colab

and restart your session.

You will be able to reuse this secret in all of your notebooks.

Please note that authentication is recommended but still optional to access public models or datasets.

```
warnings.warn(
`torch_dtype` is deprecated! Use `dtype` instead!
```

Model loaded.

1.8 Part 2: Data Preparation

Learning Problem (P): We need to format our raw CSV data into training examples.

We define helper functions to: 1. Load the CSV. 2. Build a “prompt” (Question + Options). 3. Build the full “SFT text” (Prompt + Answer) using the model’s chat template.

```
[5]: # === MCQ helpers ===
LETTER_SET = set(list("ABCDE"))

def load_mcq_dataset(csv_path: str = MCQ_CSV_PATH):
    """Load the CS189 MCQ dataset.

    Expected columns:
    - question
    - A, B, C, D, E
    - answer (single letter A-E)
    """
    df = pd.read_csv(csv_path)
    required = ["question", "A", "B", "C", "D", "E", "answer"]
    missing = [c for c in required if c not in df.columns]
    if missing:
        raise ValueError(f"Missing required columns in MCQ CSV: {missing}")

    df = df.copy()
    df["answer"] = (
        df["answer"]
        .astype(str)
        .str.strip()
        .str.upper()
    )
    df = df[df["answer"].isin(LETTER_SET)].reset_index(drop=True)
    return df

def build_mcq_prompt(row):
    """Prompt for inference: instruction + question + options.

    The model is expected to answer with the correct letter in \\boxed{} format.
```

```

"""
q = str(row["question"]).strip()
options = "\n".join([
    f"A. {row['A']}",
    f"B. {row['B']}",
    f"C. {row['C']}",
    f"D. {row['D']}",
    f"E. {row['E']}",
])
prompt = (
    "Choose exactly one correct option from A, B, C, D, and E.\n"
    "Return your answer inside a LaTeX box.\n\n"
    f"{q}\n\n{options}\n\nAnswer:"
)
return prompt

def build_mcq_sft_text(row, tokenizer):
    """
    Format a single MCQ row into the same SFT-style chat text
    as build_mmlu_sft_text.
    """
    user_content = build_mcq_prompt(row)

    # Handle answer either as "A"/"B"/... or as 0/1/2/...
    ans = row["answer"]
    if isinstance(ans, str):
        # e.g. "D"
        answer_letter = ans.strip()
    else:
        # e.g. 0-4 integer
        answer_letter = chr(ord("A") + int(ans))

    assistant_content = f"\\boxed{{{answer_letter}}}"

    messages = [
        {"role": "user", "content": user_content},
        {"role": "assistant", "content": assistant_content},
    ]

    return tokenizer.apply_chat_template(
        messages,
        tokenize=False,
        add_generation_prompt=False,
    )

```


1.8.1 Understanding the Chat Format (OpenAI Style)

To fine-tune a chat model, we need to structure our data as a conversation. This is often called the **OpenAI Chat Format** or **Messages Format**.

Instead of a single string of text, each example is a list of dictionaries, where each dictionary represents a message in the conversation: - {"role": "user", "content": "..."}: The input prompt or question. - {"role": "assistant", "content": "..."}: The model's desired response.

For our MCQ task, we structure it as: 1. **User**: "Choose exactly one correct option... [Question] ... [Options]" 2. **Assistant**: "\boxed{A}"

We then use `tokenizer.apply_chat_template()` to convert this structured list into the specific string format that the model expects (e.g., adding special tokens like `<|im_start|>user...<|im_end|>`).

```
[6]: def parse_choice_from_boxed(text: str):
    """Parse an MCQ choice A-E from the model output.

    We first look for a literal '\\boxed{X}' pattern. If not found, we
    fallback to the last standalone A-E in the decoded text.
    """
    if text is None:
        return None
    # Direct \\boxed{A} ... \\boxed{E}
    m = re.search(r"\\boxed\\{\\s*([A-E])\\s*\\}", text)
    if m:
        return m.group(1)
    # Fallback: last standalone A-E
    letters = re.findall(r"\\b([A-E])\\b", text.upper())
    if letters:
        return letters[-1]
    return None
```

1.8.2 Load and Format the (Eval) Dataset

We load the MCQ dataset and apply the formatting function. These are sample eval sets we provided. The actual test set you would be making predictions will be a mix of different questions (Full details are provided in the accompanying PDF)

```
[7]: # === Load MCQ CSV (Evaluation Data) ===
try:
    mcq_df = load_mcq_dataset(MCQ_CSV_PATH)
    print(f"Loaded MCQ dataset with {len(mcq_df)} rows from {MCQ_CSV_PATH}.")
except Exception as e:
    mcq_df = None
    print("Error loading MCQ CSV - check MCQ_CSV_PATH.")
    raise e
mcq_df
```

Loaded MCQ dataset with 25 rows from hw5_sample_eval.csv.

```
[7]:      id      question \
0      mcq_1 Peanut wants to train a model to accurately cl...
1      mcq_2 Consider a binary classification data set with...
2      mcq_3 Again, consider a binary classification data s...
3      mcq_4 Assume we are given  $X \in \mathbb{R}^{n \times d}$  and  $y \in \mathbb{R}^n$  for  $n \dots$ 
4      mcq_5 Which of the following statements are TRUE reg...
5      mcq_6 Which of the following statements are TRUE reg...
6      mcq_7 If the model resulting from Ridge regression i...
7      mcq_8 Which of the following statements are TRUE abo...
8      mcq_9 Which of the following statements are TRUE abo...
9      mcq_10 Which of the following statements are TRUE abo...
10     mcq_11 Which of the following statements are TRUE abo...
11     mcq_12 Which of the following statements are TRUE abo...
12     mcq_13 Assume a linear model  $Y = Xw^* + z$ , where  $z \in \mathbb{N} \dots$ 
13     mcq_14 How will increasing  $\lambda$  in Equation 1 affect the...
14     mcq_15 For this question,  $X \in \mathbb{R}^{n \times d}$  denotes an inpu...
15     mcq_16 Suppose you have a dataset where the label is ...
16     mcq_17 You are walking down Shattuck Ave. when you fi...
17     mcq_18 Consider the setup of the previous problem (Pr...
18     mcq_19 Consider a binary classification problem with ...
19     mcq_20 Julia is using SNE to visualize her high-dimen...
20     mcq_21 Which of the following is a problem with the s...
21     fa2025_Q3A In the bias-variance decomposition, what does ...
22     fa2025_Q3B In the bias-variance decomposition, the expres...
23     fa2025_Q3C If we reduce the L1 regularization parameter ...
24     fa2025_Q3D Suppose we change the feature basis to a const...
```

```

A \
0      High bias.
1      Close to zero.
2      The precision is 0.1, and the recall is 0.9.
3       $y = [y; 0_d]$ ,  $X = [X; \sqrt{I_d}]$ 
4      "Every entry of a matrix is non-negative" is a...
5      In the Bayesian MAP interpretation, Lasso regr...
6      Collect new data to increase the training data...
7      After a gradient descent update step, the obje...
8      The value of cross-entropy loss is always non-...
9      During the k-fold cross validation process, pr...
10     Multiplying all columns in  $X$  by a factor of tw...
11     Typically, the weights in all hidden layers ar...
12     Bias will increase.
13     Variance will increase.
14     For all  $X$  and true linear predictors  $w^* \in \mathbb{R}^d$ ,...
15     As  $n \rightarrow \infty$ ,  $\text{Acctrain}$  approaches 50%.
16     5/10
```

17 5/10
 18 Low bias implies a high F1 score.
 19 This is expected due to the nonconvexity of th...
 20 Sigmoid is prone to vanishing gradients at ext...
 21 Bias
 22 Bias
 23 Increases
 24 Remains the same

B \

0 Low bias.
 1 Close to 0.1.
 2 The precision is 0.9, and the recall is 0.1.
 3 $y = [y; 1d]$, $X = [X; \sqrt{Id}]$
 4 The singular values of a positive semi-definit...
 5 In Lasso regression, as the regularization coe...
 6 Repeat the current data twice to increase the ...
 7 There is always a unique steepest descent dire...
 8 Cross-entropy loss is only suitable for binary...
 9 During the k-fold cross validation process, pr...
 10 Multiplying the first column in X by a factor ...
 11 If using the mean squared error loss, the weig...
 12 Bias will decrease.
 13 Variance will decrease.
 14 For all X and y, $w_{ridge_2} \leq w_{OLS_2}$.
 15 As $n \rightarrow \infty$, Accval approaches 50%.
 16 between 5/10 and 9/10
 17 between 5/10 and 9/10
 18 Low variance implies a high F1 score.
 19 This is expected due to the Gaussian distribut...
 20 Sigmoid can take on negative values.
 21 $Bias^2$
 22 $Bias^2$
 23 Stays the same
 24 Increases

C \

0 High variance.
 1 Close to 0.5.
 2 The precision is 1.0, and the recall is 0.9.
 3 $y = [y; 0d]$, $X = [X; Id]$
 4 If a matrix A is positive semi-definite, then ...
 5 Lasso regression performs both feature expansi...
 6 Increase the λ_1 regularization penalty in the ...
 7 Gradient descent converges to a globally optim...
 8 For two discrete probability distributions P a...
 9 During the k-fold cross validation process, we...

10 Replacing the first column of X by the sum of ...
 11 The weight changes in a particular hidden layer...
 12 Bias will remain unchanged.
 13 Variance will remain unchanged.
 14 For all X and y, $w_{ridge_1} = w_{OLS_1}$.
 15 As $d \rightarrow \infty$, Acc_{train} approaches 100%.
 16 9/10
 17 9/10
 18 A low F1 score implies a high variance.
 19 This is expected due to the iterative nature of...
 20 Sigmoid is non-linear, which provides less representation...
 21 Variance
 22 Variance
 23 Decreases
 24 Decreases

D \

0 Low variance.
 1 Close to 0.9.
 2 The precision is 0.9, and the recall is 1.0.
 3 $y = [y; 1d]$, $X = [X; Id]$
 4 The covariance matrix of any distribution is positive semi-definite...
 5 There is no unique solution to Ridge regression...
 6 Add new features to the model.
 7 Since ReLU is a convex function, a neural network...
 8 Minimizing the cross-entropy is equivalent to minimizing the negative log-likelihood...
 9 At the end of the k-fold cross validation procedure...
 10 Not subtracting the median from X before performing PCA...
 11 Weight updates are computed in the forward pass...
 12 Decaf coldbrew latte is delicious
 13 Decaf coldbrew latte is delicious
 14 Decaf coldbrew latte is delicious
 15 As $d \rightarrow \infty$, Acc_{val} approaches 100%.
 16 more than 9/10
 17 more than 9/10
 18 A high F1 score implies low bias.
 19 Decaf coldbrew latte is delicious
 20 Sigmoid is numerically unstable when the input is close to 0 or 1...
 21 $Variance^2$
 22 $Variance^2$
 23 Cappuccino is my favorite
 24 Avocado toast

E answer

0 none of the above A
 1 Close to one. C
 2 The precision is 0.1, and the recall is 1.0. D

3	none of the above	A
4	If the Jacobian of a function is positive semi...	B
5	none of the above	B
6	Add synthetic features from the model.	A
7	none of the above	C
8	none of the above	A
9	none of the above	A
10	none of the above	B
11	none of the above	B
12	none of the above	A
13	none of the above	B
14	none of the above	C
15	none of the above	D
16	none of the above	C
17	none of the above	B
18	none of the above	D
19	none of the above	A
20	none of the above	A
21	Noise	E
22	Cold brew latte is delicious	C
23	The moon is made of cheese	C
24	Matcha cream puff	C

```
[8]: def build_mcq_eval_dataset(mcq_df, tokenizer):
      """
      Turn the MCQ pandas DataFrame into a Hugging Face Dataset
      with a single 'text' column formatted like the MMLU SFT data.
      """
      # Convert pandas DataFrame to datasets.Dataset
      ds = Dataset.from_pandas(mcq_df.reset_index(drop=True))

      def _format_example(example):
          return {
              "text": build_mcq_sft_text(example, tokenizer)
          }

      ds = ds.map(
          _format_example,
          remove_columns=ds.column_names,  # keep only 'text'
      )
      return ds
```

```
[9]: eval_dataset = build_mcq_eval_dataset(mcq_df, tokenizer)
      print(eval_dataset[0]["text"])  # sanity check
```

Map: 0% | 0/25 [00:00<?, ? examples/s]

<|im_start|>system

You are Qwen, created by Alibaba Cloud. You are a helpful assistant.<|im_end|>
<|im_start|>user
Choose exactly one correct option from A, B, C, D, and E.
Return your answer inside a LaTeX box.

Peanut wants to train a model to accurately classify different types of animals from images. After training and testing his model, he observes that the model has high training error and high test error. What can we most confidently say about the bias/variance characteristics of Peanut's model?

- A. High bias.
- B. Low bias.
- C. High variance.
- D. Low variance.
- E. none of the above

Answer:<|im_end|>
<|im_start|>assistant
\boxed{A}<|im_end|>

1.8.3 Load Training Dataset (MMLU)

We will use the MMLU (Massive Multitask Language Understanding) dataset, specifically the machine_learning subset, as our training data. This helps the model learn general machine learning concepts which should transfer to the CS189 exam problems.

```
[10]: # === MMLU Helper Functions ===
def load_mmlu_dataset(subset: str = "machine_learning", split: str = "test"):
    """Load a subset of the MMLU dataset from Hugging Face."""
    print(f>Loading MMLU dataset (subset={subset}, split={split})...)
    ds = load_dataset("cais/mmlu", subset, split=split)
    return ds

def build_mmlu_prompt(row):
    """Prompt for inference: instruction + question + options."""
    q = str(row["question"]).strip()
    choices = row["choices"]

    options_list = []
    for i, choice in enumerate(choices):
        letter = chr(ord("A") + i)
        options_list.append(f"{letter}. {choice}")
    options_str = "\n".join(options_list)

    prompt = (
        "Choose exactly one correct option from the choices provided.\n"
        "Return your answer inside a LaTeX box.\n\n"
```

```

        f"{q}\n\n{options_str}\n\nAnswer:"
    )
    return prompt

def build_mmlu_sft_text(row, tokenizer):
    """Build properly formatted chat template text for training."""
    user_content = build_mmlu_prompt(row)

    answer_int = row["answer"]
    answer_letter = chr(ord("A") + answer_int)
    assistant_content = f"\boxed{{{answer_letter}}}"

    messages = [
        {"role": "user", "content": user_content},
        {"role": "assistant", "content": assistant_content}
    ]

    return tokenizer.apply_chat_template(
        messages,
        tokenize=False,
        add_generation_prompt=False
    )

# === Load MMLU Machine Learning Dataset ===
mmlu_ds = load_mmlu_dataset("machine_learning", split="test")
mmlu_text_ds = mmlu_ds.map(lambda x: {"text": build_mmlu_sft_text(x,
↪tokenizer)})
print("Loaded MMLU ML dataset with", len(mmlu_text_ds), "rows")

# Set the training dataset - you can mix and match datasets here
train_dataset = mmlu_text_ds

```

Loading MMLU dataset (subset=machine_learning, split=test)...

Loaded MMLU ML dataset with 112 rows

Let's look at an example of the training data to see what the model is actually seeing as input. Note that there are now start and stop tokens `<|im_start|>` and `<|im_end|>` as well as the role user and assistant indicating who is speaking.

```

[11]: # print out what the first row looks like
print(train_dataset[0]['text'])

```

```

<|im_start|>system
You are Qwen, created by Alibaba Cloud. You are a helpful assistant.<|im_end|>
<|im_start|>user
Choose exactly one correct option from the choices provided.
Return your answer inside a LaTeX box.

```

Statement 1| Linear regression estimator has the smallest variance among all unbiased estimators. Statement 2| The coefficients assigned to the classifiers assembled by AdaBoost are always non-negative.

- A. True, True
- B. False, False
- C. True, False
- D. False, True

Answer:<|im_end|>
<|im_start|>assistant
\boxed{D}<|im_end|>

1.9 Part 3: Baseline Evaluation

Predict & Evaluate (O): Before we train, let's see how the model performs "zero-shot" or "few-shot" (depending on the prompt) on our CS189 questions.

```
[12]: def eval_mcq_accuracy(
    curr_model,
    curr_tokenizer,
    df,
    max_new_tokens: int = 64,
    return_details: bool = False,
):
    """Evaluate a model on the MCQ dataset using greedy decoding.

    If return_details=True, also return a pandas DataFrame with
    [idx, question, A, B, C, D, E, gold, decoded, parsed, correct].
    """

    curr_model.eval()
    n = len(df)
    correct = 0
    total = 0
    records = []

    for idx in range(n):
        row = df.iloc[idx]
        user_content = build_mcq_prompt(row)

        # Apply chat template for inference
        messages = [{"role": "user", "content": user_content}]
        prompt = curr_tokenizer.apply_chat_template(
            messages,
            tokenize=False,
```



```

        add_generation_prompt=True
    )

    inputs = curr_tokenizer(prompt, return_tensors="pt").to(device)

    with torch.no_grad():
        outputs = curr_model.generate(
            **inputs,
            max_new_tokens=max_new_tokens,
            do_sample=False,
        )

    gen_tokens = outputs[0][inputs["input_ids"].shape[1]:]
    decoded = curr_tokenizer.decode(gen_tokens, skip_special_tokens=True)

    pred = parse_choice_from_boxed(decoded)
    is_correct = (pred is not None and pred == row["answer"])
    if is_correct:
        correct += 1
    total += 1

    records.append({
        "idx": idx,
        "question": row["question"],
        "A": row["A"],
        "B": row["B"],
        "C": row["C"],
        "D": row["D"],
        "E": row["E"],
        "gold": row["answer"],
        "prompt": prompt,
        "decoded": decoded,
        "parsed": pred,
        "correct": is_correct,
    })

    if (idx + 1) % 20 == 0:
        print(f"Processed {idx + 1}/{n} questions...")

acc = correct / max(total, 1)
print(f"MCQ accuracy: {acc * 100:.2f}% ({correct}/{total}")

details_df = pd.DataFrame(records)
if return_details:
    return acc, details_df
return acc

```

```
[13]: # === Baseline MCQ accuracy before fine-tuning ===
print("Evaluating baseline model on MCQ dataset...")
baseline_acc, baseline_details = eval_mcq_accuracy(
    model,
    tokenizer,
    mcq_df,
    max_new_tokens=EVAL_MAX_NEW_TOKENS,
    return_details=True,
)
baseline_details.head()
```

The following generation flags are not valid and may be ignored: ['temperature', 'top_p', 'top_k']. Set `TRANSFORMERS\_VERBOSITY=info` for more details.

Evaluating baseline model on MCQ dataset...

Processed 20/25 questions...

MCQ accuracy: 28.00% (7/25)

```
[13]:      idx      question \
0      0  Peanut wants to train a model to accurately cl...
1      1  Consider a binary classification data set with...
2      2  Again, consider a binary classification data s...
3      3  Assume we are given  $X \in \mathbb{R}^{n \times d}$  and  $y \in \mathbb{R}^n$  for  $n \dots$ 
4      4  Which of the following statements are TRUE reg...

                                     A \
0                                     High bias.
1                                     Close to zero.
2      The precision is 0.1, and the recall is 0.9.
3       $y = [y; 0_d]$ ,  $X = [X; \sqrt{I_d}]$ 
4  "Every entry of a matrix is non-negative" is a...

                                     B \
0                                     Low bias.
1                                     Close to 0.1.
2      The precision is 0.9, and the recall is 0.1.
3       $y = [y; I_d]$ ,  $X = [X; \sqrt{I_d}]$ 
4  The singular values of a positive semi-definit...

                                     C \
0                                     High variance.
1                                     Close to 0.5.
2      The precision is 1.0, and the recall is 0.9.
3       $y = [y; 0_d]$ ,  $X = [X; I_d]$ 
4  If a matrix A is positive semi-definite, then ...

                                     D \
0                                     Low variance.
```

```

1                                Close to 0.9.
2      The precision is 0.9, and the recall is 1.0.
3                                y = [ y; 1d ], X = [ X; 1d ]
4  The covariance matrix of any distribution is p...

                                E gold \
0                                none of the above    A
1                                Close to one.        C
2      The precision is 0.1, and the recall is 1.0.    D
3                                none of the above    A
4  If the Jacobian of a function is positive semi...  B

                                prompt \
0 <|im_start|>system\nYou are Qwen, created by A...
1 <|im_start|>system\nYou are Qwen, created by A...
2 <|im_start|>system\nYou are Qwen, created by A...
3 <|im_start|>system\nYou are Qwen, created by A...
4 <|im_start|>system\nYou are Qwen, created by A...

                                decoded parsed correct
0  To determine what we can most confidently say ...    A    True
1  To determine the area under the ROC curve (AUC...    A    False
2  To determine the precision and recall for a cl...    A    False
3  To determine which modified version of \(\mathcal{X}\) a...    A    True
4  To determine which statement is true regarding...    A    False

```

1.10 Part 4: Training (Optimization)

Optimization (M): We now configure the SFTTrainer.

We set: - dataset_text_field="text": Tells the trainer which column contains the formatted chat. - learning_rate, batch_size: Standard hyperparameters. - optim="adamw_8bit": Efficient optimizer.

```

[14]: # === Set up SFTTrainer ===
sft_config = SFTConfig(
    dataset_text_field="text",
    per_device_train_batch_size=TRAIN_BATCH_SIZE,
    gradient_accumulation_steps=GRADIENT_ACCUMULATION_STEPS,
    warmup_steps=WARMUP_STEPS,
    max_steps=MAX_STEPS,
    learning_rate=LEARNING_RATE,
    logging_steps=1,
    optim=OPTIM,
    weight_decay=WEIGHT_DECAY,
    lr_scheduler_type=LR_SCHEDULER_TYPE,

```

```

seed=SEED,
report_to="none",

# === CHECKPOINTING EVERY 10 STEPS ===
save_strategy="steps",
save_steps=10,
save_total_limit=3,

# === EVALUATION EVERY 10 STEPS ===
eval_strategy="steps",
eval_steps=10,

# === BEST MODEL SELECTION ===
load_best_model_at_end=True,
metric_for_best_model="loss", # TRL uses "loss" instead of "eval_loss"
greater_is_better=False,
)

trainer = SFTTrainer(
    model=model,
    args=sft_config,
    train_dataset=train_dataset,
    eval_dataset=eval_dataset,
    processing_class=tokenizer,
)

trainer

```

```

Adding EOS to eval dataset:  0%|          | 0/25 [00:00<?, ? examples/s]
Tokenizing eval dataset:    0%|          | 0/25 [00:00<?, ? examples/s]
Truncating eval dataset:    0%|          | 0/25 [00:00<?, ? examples/s]

The model is already on multiple devices. Skipping the move to device specified
in `args`.

```

[14]: <trl.trainer.sft_trainer.SFTTrainer at 0x7bd7c83e2c00>

1.10.1 Run Training

This will iterate through the dataset and update the model's weights.

```

[15]: # === Fine-tune the model ===
model.train()
trainer.train()
model.eval()

```

The tokenizer has new PAD/BOS/EOS tokens that differ from the model config and generation config. The model config and generation config were aligned accordingly, being updated with the tokenizer's values. Updated tokens: {'bos_token_id': None, 'pad_token_id': 151643}.

<IPython.core.display.HTML object>

There were missing keys in the checkpoint model loaded: ['lm_head.weight'].

```
[15]: Qwen2ForCausalLM(
  (model): Qwen2Model(
    (embed_tokens): Embedding(151665, 896)
    (layers): ModuleList(
      (0-23): 24 x Qwen2DecoderLayer(
        (self_attn): Qwen2Attention(
          (q_proj): Linear(in_features=896, out_features=896, bias=True)
          (k_proj): Linear(in_features=896, out_features=128, bias=True)
          (v_proj): Linear(in_features=896, out_features=128, bias=True)
          (o_proj): Linear(in_features=896, out_features=896, bias=False)
        )
        (mlp): Qwen2MLP(
          (gate_proj): Linear(in_features=896, out_features=4864, bias=False)
          (up_proj): Linear(in_features=896, out_features=4864, bias=False)
          (down_proj): Linear(in_features=4864, out_features=896, bias=False)
          (act_fn): SiLUActivation()
        )
        (input_layernorm): Qwen2RMSNorm((896,), eps=1e-06)
        (post_attention_layernorm): Qwen2RMSNorm((896,), eps=1e-06)
      )
    )
    (norm): Qwen2RMSNorm((896,), eps=1e-06)
    (rotary_emb): Qwen2RotaryEmbedding()
  )
  (lm_head): Linear(in_features=896, out_features=151665, bias=False)
)
```

1.11 Part 5: Post-Training Evaluation

Predict & Evaluate (O): Now that the model is trained, we evaluate it again on the same MCQ dataset to see if accuracy improved.

```
[16]: # === Evaluate MCQ accuracy after fine-tuning ===
print("Evaluating fine-tuned model on MCQ dataset...")
ft_acc, ft_details = eval_mcq_accuracy(
    model,
    tokenizer,
    mcq_df,
```

```

    max_new_tokens=EVAL_MAX_NEW_TOKENS,
    return_details=True,
)
ft_details.head()
print(f"Baseline acc: {baseline_acc:.4f}, Fine-tuned acc: {ft_acc:.4f}")

```

Evaluating fine-tuned model on MCQ dataset...

Processed 20/25 questions...

MCQ accuracy: 16.00% (4/25)

Baseline acc: 0.2800, Fine-tuned acc: 0.1600

1.11.1 Save the Model

We save the fine-tuned model and tokenizer so we can use them later.

```

[17]: # === Save fine-tuned model (optional) ===
os.makedirs(OUTPUT_DIR, exist_ok=True)
trainer.model.save_pretrained(OUTPUT_DIR)
tokenizer.save_pretrained(OUTPUT_DIR)
print("Saved fine-tuned model to", OUTPUT_DIR)

```

Saved fine-tuned model to ./mcq_finetuned_model

1.12 YOUR TURN — Final Kaggle Submission

Now it's your turn to run the *full* ML lifecycle.

You should now be able to load and inspect the **private test CSV**, which contains **169 questions** with mixed types.

Use your **fine-tuned LLM** to make predictions on these questions.

Beware of formatting: Kaggle will reject incorrectly formatted submissions!

1.12.1 Objective

Your task is to:

1. **Use the provided notebook** to fine-tune the base model on your own choice of train data (feel free to adapt the one we provided)
2. **Adapt the same pipeline** to run inference on `test.csv`.
3. For each row in `test.csv`, output **exactly one letter** from the set **{A, B, C, D, E}**.
4. Save these predictions in the **strict submission format** described below and upload your CSV to Kaggle:
 - <https://www.kaggle.com/t/11c8ffdc967fe3f27755cde6fb5810e8>

1.12.2 Submission Format (Strict)

Your submission must be a **CSV** with exactly **two columns** — `id` and `prediction` — and a **single header row**.

A valid submission looks like:

id	prediction
test_00001	A
test_00002	A
test_00003	C
test_00004	E
...	...

1.12.3 Evaluation Metric

Submissions are evaluated using **accuracy**: the fraction of test examples for which your predicted answer matches the hidden correct answer.

1.12.4 Example

id	True Answer	Your Prediction	Correct?
test_00001	A	A	Yes
test_00002	A	B	No
test_00003	A	A	Yes

Here, the accuracy would be 2/3, or approximately 66.7%

The leaderboard is split into:

- **Public leaderboard**: 50% of the test data
- **Private leaderboard**: remaining 50% (used for final ranking and grading)

```
[29]: # === Load base model & tokenizer ===
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)

# Ensure we have a pad token for training
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

model = AutoModelForCausalLM.from_pretrained(
    MODEL_NAME,
    torch_dtype=torch.float16 if torch.cuda.is_available() else torch.float32,
    device_map="auto" if torch.cuda.is_available() else None,
)

model.resize_token_embeddings(len(tokenizer))
model.to(device)
model.eval()
print('Model loaded.')
```

Model loaded.

```
[42]: def load_mmlu_dataset_five_options(subset: str = "machine_learning", split: str =
      ↪ "test"):
      print(f>Loading MMLU dataset (subset={subset}, split={split})...)
      ds = load_dataset("cais/mmlu", subset, split=split)
      ds = ds.cast_column("answer", Value("int64"))
      return ds

def add_option_E(ds, p_flip: float = 0.2, seed: int = SEED):
    rng = random.Random(seed)

    def _map(example):
        choices = list(example["choices"])
        answer = int(example["answer"])
        if len(choices) == 4:
            choices.append("None of the answers.")

            # With probability p_flip, flip correct answer to E
            if rng.random() < p_flip:
                true_answer_text = choices[answer]
                choices[answer] = "None of the answers."
                choices[4] = true_answer_text
                answer = 4

        return {
            "choices": choices,
            "answer": answer,
        }

    return ds.map(_map)

def prepare_mmlu_dataset(subset: str = "machine_learning", split: str =
      ↪ "test", sample_size = None):
    ds = load_mmlu_dataset_five_options(subset, split)
    if sample_size is not None:
        ds = ds.shuffle(seed=SEED).select(range(sample_size))
    ds = add_option_E(ds, p_flip=0.2, seed=SEED)
    ds = ds.map(lambda x: {"text": build_mmlu_sft_text(x, tokenizer)})
    ds = ds.shuffle(seed=SEED)
    return ds

def prepare_mmlu_dataset_rag(subset: str = "machine_learning", split: str =
      ↪ "test", sample_size = None):
    ds = load_mmlu_dataset_five_options(subset, split)
    if sample_size is not None:
        ds = ds.shuffle(seed=SEED).select(range(sample_size))
```



```

ds = add_option_E(ds, p_flip=0.2, seed=SEED)
ds = ds.map(lambda x: {"text": build_mmlu_sft_text_rag(x, tokenizer)})
ds = ds.shuffle(seed=SEED)
return ds

```

```

[31]: mmlu_ds = prepare_mmlu_dataset()
specialised_dataset = mmlu_ds

```

Loading MMLU dataset (subset=machine_learning, split=test)...

```

[32]: # =====
# === CONFIGURATION - ALL SETTINGS IN ONE PLACE ===
# =====

# --- Model Configuration ---
MODEL_NAME = "Qwen/Qwen2.5-0.5B-Instruct" # YOU CANNOT CHANGE THIS

# --- Dataset Configuration ---
#TODO: REPLACE WITH YOUR OWN PATH
MCQ_CSV_PATH = "hw5_sample_eval.csv" # Path to CS189 MCQ sample eval dataset

# --- Training Configuration (feel free to adjust!) ---
TRAIN_BATCH_SIZE = 1 # how many examples the device sees at one time
GRADIENT_ACCUMULATION_STEPS = 4 # over how many examples is loss accumulated
    ↳ before 1 optimiser update(step) is performed
WARMUP_STEPS = 5 # how many optimiser updates are ignored at the start (noisy/
    ↳ unstable gradients)
MAX_STEPS = 50 # how many times will optimiser be updated (epochs will be of
    ↳ less priority)
LEARNING_RATE = 1e-5
WEIGHT_DECAY = 0.01
LR_SCHEDULER_TYPE = "cosine" # after warm-up steps, how will LR decay to 0 over
    ↳ remaining steps
OPTIM = "adamw_8bit" # requires bitsandbytes
SEED = 189

# --- Evaluation Configuration ---
EVAL_MAX_NEW_TOKENS = 64 # How many tokens to generate for inference
OUTPUT_DIR = "./mcq_finetuned_model"

```

```

[33]: sft_stage1_config = SFTConfig(
    dataset_text_field="text",
    per_device_train_batch_size=TRAIN_BATCH_SIZE,
    gradient_accumulation_steps=GRADIENT_ACCUMULATION_STEPS,
    warmup_steps=WARMUP_STEPS,
    max_steps=MAX_STEPS,
    learning_rate=LEARNING_RATE,

```

```

logging_steps=5,
optim=OPTIM,
weight_decay=WEIGHT_DECAY,
lr_scheduler_type=LR_SCHEDULER_TYPE,
seed=SEED,
report_to="none",

# === CHECKPOINTING EVERY 10 STEPS ===
save_strategy="steps",
save_steps=10,
save_total_limit=3,

# === EVALUATION EVERY 10 STEPS ===
eval_strategy="steps",
eval_steps=5,

# === BEST MODEL SELECTION ===
load_best_model_at_end=True,
metric_for_best_model="eval_loss",
greater_is_better=False,
)
trainer_1 = SFTTrainer(
    model=model,
    train_dataset=specialised_dataset,
    eval_dataset=eval_dataset,
    args=sft_stage1_config,
    processing_class=tokenizer,
)

train_result = trainer_1.train()
trainer_1.save_model("qwen_dmt_stage1_mmlu")
tokenizer.save_pretrained("qwen_dmt_stage1_mmlu")

# === Plot training loss vs steps ===
log_history = trainer_1.state.log_history

steps = [entry["step"] for entry in log_history if "loss" in entry]
losses = [entry["loss"] for entry in log_history if "loss" in entry]
eval_losses = [entry["eval_loss"] for entry in log_history if "eval_loss" in
    ↪entry]

plt.figure(figsize=(8,5))
plt.plot(steps, losses, label="Train Loss", marker="o")
plt.plot(steps, eval_losses, label="Eval Loss", marker="x")
plt.xlabel("Training step")
plt.ylabel("Loss")

```

```
plt.title("Training vs Eval Loss (Stage 1)")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

Adding EOS to train dataset: 0%| | 0/112 [00:00<?, ? examples/s]

Tokenizing train dataset: 0%| | 0/112 [00:00<?, ? examples/s]

Truncating train dataset: 0%| | 0/112 [00:00<?, ? examples/s]

Adding EOS to eval dataset: 0%| | 0/25 [00:00<?, ? examples/s]

Tokenizing eval dataset: 0%| | 0/25 [00:00<?, ? examples/s]

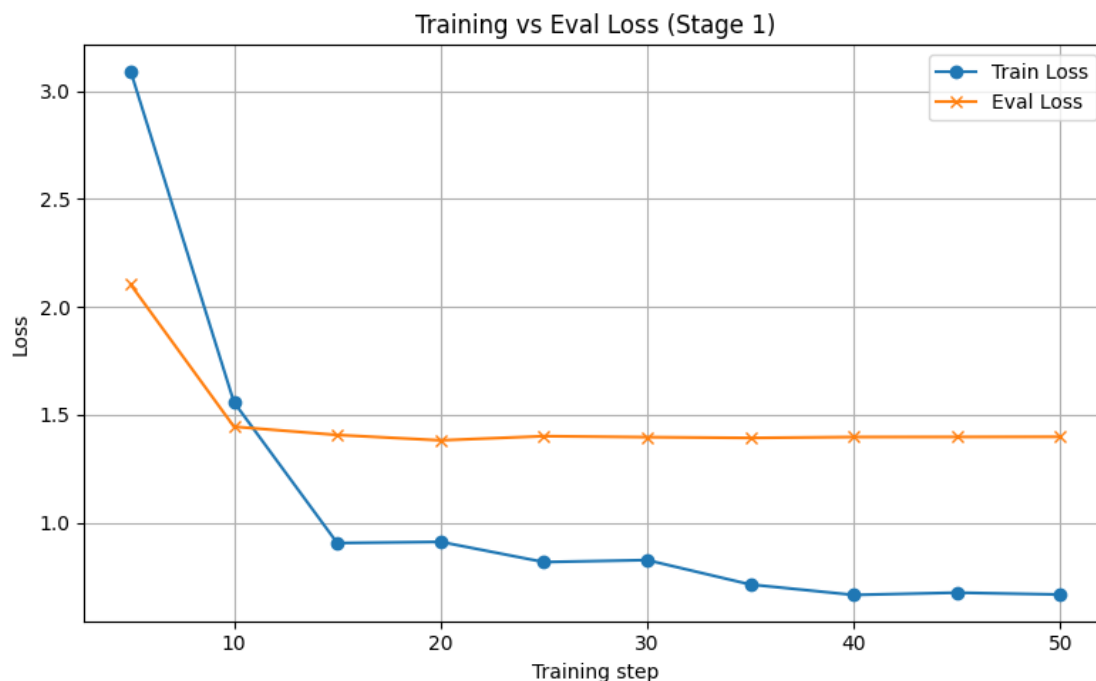
Truncating eval dataset: 0%| | 0/25 [00:00<?, ? examples/s]

The model is already on multiple devices. Skipping the move to device specified in `args`.

The tokenizer has new PAD/BOS/EOS tokens that differ from the model config and generation config. The model config and generation config were aligned accordingly, being updated with the tokenizer's values. Updated tokens: {'bos_token_id': None, 'pad_token_id': 151643}.

<IPython.core.display.HTML object>

There were missing keys in the checkpoint model loaded: ['lm_head.weight'].



```
[34]: print("Evaluating fine-tuned model on MCQ dataset...")
finetuned_acc, finetuned_details = eval_mcq_accuracy(
    model,
    tokenizer,
    mcq_df,
    max_new_tokens=EVAL_MAX_NEW_TOKENS,
    return_details = True
)
finetuned_details.head()
```

Evaluating fine-tuned model on MCQ dataset...

Processed 20/25 questions...

MCQ accuracy: 40.00% (10/25)

```
[34]:      idx      question \
0      0  Peanut wants to train a model to accurately cl...
1      1  Consider a binary classification data set with...
2      2  Again, consider a binary classification data s...
3      3  Assume we are given  $X \in \mathbb{R}^{n \times d}$  and  $y \in \mathbb{R}^n$  for  $n \dots$ 
4      4  Which of the following statements are TRUE reg...

                                     A \
0                                     High bias.
1                                     Close to zero.
2      The precision is 0.1, and the recall is 0.9.
3       $y = [y; 0_d]$ ,  $X = [X; \sqrt{I_d}]$ 
4  "Every entry of a matrix is non-negative" is a...

                                     B \
0                                     Low bias.
1                                     Close to 0.1.
2      The precision is 0.9, and the recall is 0.1.
3       $y = [y; 1_d]$ ,  $X = [X; \sqrt{I_d}]$ 
4  The singular values of a positive semi-definit...

                                     C \
0                                     High variance.
1                                     Close to 0.5.
2      The precision is 1.0, and the recall is 0.9.
3       $y = [y; 0_d]$ ,  $X = [X; I_d]$ 
4  If a matrix A is positive semi-definite, then ...

                                     D \
0                                     Low variance.
1                                     Close to 0.9.
2      The precision is 0.9, and the recall is 1.0.
3       $y = [y; 1_d]$ ,  $X = [X; I_d]$ 
```

```

4 The covariance matrix of any distribution is p...

                                E gold \
0                                none of the above A
1                                Close to one. C
2 The precision is 0.1, and the recall is 1.0. D
3                                none of the above A
4 If the Jacobian of a function is positive semi... B

                                prompt   decoded parsed \
0 <|im_start|>system\nYou are Qwen, created by A... \boxed{C} C
1 <|im_start|>system\nYou are Qwen, created by A... \boxed{E} E
2 <|im_start|>system\nYou are Qwen, created by A... \boxed{B} B
3 <|im_start|>system\nYou are Qwen, created by A... \boxed{B} B
4 <|im_start|>system\nYou are Qwen, created by A... \boxed{B} B

correct
0 False
1 False
2 False
3 False
4 True

```

```

[43]: # ===== RAG HELPER FUNCTIONS =====
def build_rag_corpus_from_mmlu(ds):
    """
    Turn each MMLU example into a text chunk that includes:
    - the question
    - all choices
    - the correct answer in plain text
    """
    def _format(example):
        question = example["question"]
        choices = example["choices"]
        answer_idx = int(example["answer"])
        correct_choice_letter = chr(ord("A") + answer_idx)
        correct_choice_text = choices[answer_idx]

        choices_block = "\n".join(
            f"{chr(ord('A') + i)}. {c}" for i, c in enumerate(choices)
        )

        text = (
            f"Question: {question}\n"
            f"Choices:\n{choices_block}\n\n"
            f"Correct answer: {correct_choice_letter}. {correct_choice_text}\n"
        )

```

```

        return {"corpus_text": text}

rag_corpus = ds.map(_format)
return rag_corpus

def build_rag_query_from_row(row) -> str:
    """Use question + options as retrieval query."""
    q = str(row["question"]).strip()
    choices_str = " ".join([
        f"A. {row['A']}",
        f"B. {row['B']}",
        f"C. {row['C']}",
        f"D. {row['D']}",
        f"E. {row['E']}",
    ])
    return f"{q} {choices_str}"

def retrieve_contexts_for_row(row, top_k: int = 2):
    """Retrieve top-k corpus_text chunks for this MCQ row."""
    query = build_rag_query_from_row(row)

    q_emb = embed_model.encode(
        [query],
        convert_to_numpy=True,
        normalize_embeddings=True,
    )[0]

    scores, retrieved = rag_corpus.get_nearest_examples(
        "embedding",
        q_emb,
        k=top_k,
    )
    contexts = retrieved["corpus_text"] # list[str]
    return contexts, scores

def build_rag_user_content(row, contexts):
    base = build_mcq_prompt(row)

    context_block = ""
    for i, ctx in enumerate(contexts, 1):
        context_block += f"[CONTEXT {i}]\n{ctx}\n\n"

    user_content = (
        "Use the following context if it is relevant to the question.\n"

```

```

        f"{context_block}"
        f"{base}"
    )
    return user_content

def mmlu_example_to_row(example):
    """Adapt an MMLU example to the 'row' format expected by your RAG helpers.
    ↪ """
    choices = example["choices"]
    row = {
        "question": example["question"],
        "A": choices[0],
        "B": choices[1],
        "C": choices[2],
        "D": choices[3],
        "E": choices[4],
        "answer": example["answer"],
    }
    return row

def build_mmlu_sft_text_rag(example, tokenizer, top_k: int = 2, p_no_context:
    ↪ float = 0.1):
    """
    RAG-aware version of build_mmlu_sft_text.

    Uses your existing:
    - mmlu_example_to_row
    - retrieve_contexts_for_row
    - build_rag_user_content
    """
    import random

    row = mmlu_example_to_row(example)

    if random.random() < p_no_context:
        contexts = []
    else:
        contexts, _ = retrieve_contexts_for_row(row, top_k=top_k)

    user_content = build_rag_user_content(row, contexts)

    ans = example["answer"]
    if isinstance(ans, str):
        answer_letter = ans.strip().upper()
    else:
        answer_letter = chr(ord("A") + int(ans))

```

```

assistant_content = f"\\boxed{{{answer_letter}}}"

messages = [
    {"role": "user", "content": user_content},
    {"role": "assistant", "content": assistant_content},
]

text = tokenizer.apply_chat_template(
    messages,
    tokenize=False,
    add_generation_prompt=False,
)

return text

```

```

[36]: rag_corpus = None
corpus_subset = load_mmlu_dataset_five_options(subset="all",split="test")
corpus_subset = add_option_E(corpus_subset,p_flip=0.2,seed=SEED)
rag_corpus = build_rag_corpus_from_mmlu(corpus_subset)

```

Loading MMLU dataset (subset=all, split=test)...

```

[39]: from sentence_transformers import SentenceTransformer

embed_model_name = "BAAI/bge-small-en-v1.5"
embed_model = SentenceTransformer(embed_model_name)

def embed_batch(batch):
    # batch["corpus_text"] is a list of strings
    embeddings = embed_model.encode(
        batch["corpus_text"],
        batch_size=64,
        convert_to_numpy=True,
        show_progress_bar=False,
        normalize_embeddings=True,
    )
    batch["embedding"] = embeddings.tolist()
    return batch

# Add embedding column
rag_corpus = rag_corpus.map(
    embed_batch,
    batched=True,
    batch_size=64
)

# Tell Datasets that "embedding" is a FAISS indexable column

```



```
rag_corpus = rag_corpus.add_faiss_index(column="embedding")
```

```
Map: 0%|          | 0/14042 [00:00<?, ? examples/s]
```

```
0%|          | 0/15 [00:00<?, ?it/s]
```

```
[44]: def predict_mcq_test_with_rag(
    curr_model,
    curr_tokenizer,
    df,
    max_new_tokens: int = 64,
    return_details: bool = False,
    top_k: int = 2,
):
    """
    Use the model + RAG to make predictions on an MCQ test dataframe.
    """
    curr_model.eval()
    n = len(df)
    records = []

    for idx in range(n):
        row = df.iloc[idx]

        contexts, _ = retrieve_contexts_for_row(row, top_k=top_k)
        user_content = build_rag_user_content(row, contexts)

        messages = [{"role": "user", "content": user_content}]
        prompt = curr_tokenizer.apply_chat_template(
            messages,
            tokenize=False,
            add_generation_prompt=True,
        )

        inputs = curr_tokenizer(prompt, return_tensors="pt").to(device)

        with torch.no_grad():
            outputs = curr_model.generate(
                **inputs,
                max_new_tokens=max_new_tokens,
                do_sample=False,
                pad_token_id=curr_tokenizer.eos_token_id,
            )

            gen_tokens = outputs[0][inputs["input_ids"].shape[1]:]
            decoded = curr_tokenizer.decode(gen_tokens, skip_special_tokens=True)

            pred = parse_choice_from_boxed(decoded)
```

```

    if pred is None:
        pred = "A"

    records.append({
        "id": row["id"],
        "question": row["question"],
        "A": row["A"],
        "B": row["B"],
        "C": row["C"],
        "D": row["D"],
        "E": row["E"],
        "prompt": prompt,
        "decoded": decoded,
        "parsed": pred,
    })

    if (idx + 1) % 20 == 0:
        print(f"Processed {idx + 1}/{n} questions...")

details_df = pd.DataFrame(records)

submission_df = details_df[["id", "parsed"]].rename(
    columns={"parsed": "prediction"}
)

if return_details:
    return submission_df, details_df
return submission_df

```

```

[27]: # def predict_mcq_test(
#     curr_model,
#     curr_tokenizer,
#     df,
#     max_new_tokens: int = 64,
#     return_details: bool = False,
# ):
#     """
#     Use the model to make predictions on an MCQ test dataframe
#
#     Returns:
#         submission_df: DataFrame with columns ['id', 'prediction']
#                        (letter A-E), ready for Kaggle.
#         details_df (optional): if return_details=True, a DataFrame with
#                                extra columns for inspection.
#     """
#     curr_model.eval()
#     n = len(df)

```

```

#     records = []

#     for idx in range(n):
#         row = df.iloc[idx]
#         user_content = build_mcq_prompt(row)

#         # Apply chat template for inference
#         messages = [{"role": "user", "content": user_content}]
#         prompt = curr_tokenizer.apply_chat_template(
#             messages,
#             tokenize=False,
#             add_generation_prompt=True,
#         )

#         inputs = curr_tokenizer(prompt, return_tensors="pt").to(device)

#         with torch.no_grad():
#             outputs = curr_model.generate(
#                 **inputs,
#                 max_new_tokens=max_new_tokens,
#                 do_sample=False,
#             )

#         gen_tokens = outputs[0][inputs["input_ids"].shape[1]:]
#         decoded = curr_tokenizer.decode(gen_tokens, skip_special_tokens=True)

#         pred = parse_choice_from_boxed(decoded)

#         if pred is None:
#             pred = "A"

#         records.append({
#             "id": row["id"],
#             "question": row["question"],
#             "A": row["A"],
#             "B": row["B"],
#             "C": row["C"],
#             "D": row["D"],
#             "E": row["E"],
#             "prompt": prompt,
#             "decoded": decoded,
#             "parsed": pred,
#         })

#     if (idx + 1) % 20 == 0:
#         print(f"Processed {idx + 1}/{n} questions...")

```

```

#     details_df = pd.DataFrame(records)

#     submission_df = details_df[["id", "parsed"]].rename(
#         columns={"parsed": "prediction"}
#     )

#     if return_details:
#         return submission_df, details_df
#     return submission_df

```

```

[41]: YOUR_PATH_TO_TEST_CSV = 'kaggle_test.csv' #TODO: REPLACE WITH YOUR OWN PATH
test_questions = pd.read_csv(YOUR_PATH_TO_TEST_CSV)
test_questions
#TODO:
# 1. Make predictions on your finetuned model
submission_df, debug_df = predict_mcq_test_with_rag(
    curr_model=model,
    curr_tokenizer=tokenizer,
    df=test_questions,
    max_new_tokens=64,
    return_details=True,
)
# 2. submit to kaggle following the expected format (id, prediction)
# Save to CSV
submission_df.to_csv("submission.csv", index=False)

print("Saved submission.csv")
submission_df.head()

```

```

Processed 20/169 questions...
Processed 40/169 questions...
Processed 60/169 questions...
Processed 80/169 questions...
Processed 100/169 questions...
Processed 120/169 questions...
Processed 140/169 questions...
Processed 160/169 questions...
Saved submission.csv

```

```

[41]:      id prediction
0  test_00000         C
1  test_00001         E
2  test_00002         A
3  test_00003         B
4  test_00004         B

```

```
[54]: # CATASTROPHIC FORGETTING
# === Load base model & tokenizer ===
baseline_tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)

# Ensure we have a pad token for training
if baseline_tokenizer.pad_token is None:
    baseline_tokenizer.pad_token = baseline_tokenizer.eos_token

baseline_model = AutoModelForCausalLM.from_pretrained(
    MODEL_NAME,
    torch_dtype=torch.float16 if torch.cuda.is_available() else torch.float32,
    device_map="auto" if torch.cuda.is_available() else None,
)

baseline_model.resize_token_embeddings(len(baseline_tokenizer))
baseline_model.to(device)
baseline_model.eval()
print('Baseline Model loaded.')

def run_chat(model, tokenizer, user_content, max_new_tokens=128):
    message = [{"role": "user", "content": user_content}]
    prompt = tokenizer.apply_chat_template(
        message,
        tokenize=False,
        add_generation_prompt=True,
    )
    inputs = tokenizer(prompt, return_tensors="pt").to(device)

    with torch.no_grad():
        outputs = model.generate(
            **inputs,
            max_new_tokens=max_new_tokens,
            do_sample=False,
            pad_token_id=tokenizer.eos_token_id,
        )

    gen_tokens = outputs[0][inputs["input_ids"].shape[1]:]
    decoded = tokenizer.decode(gen_tokens, skip_special_tokens=True)
    return decoded.strip()

examples = [
    # Q3: catastrophic forgetting
    {
        "id": "Q3_1",
        "type": "catastrophic_forgetting",
        "input": "If it rains, I bring an umbrella out. So if I do not bring an  

        ↪umbrella out, it must mean it is not raining. Is this true?",
    }
]
```

```

    },
    {
        "id": "Q3_2",
        "type": "catastrophic_forgetting",
        "input": "What is the powerhouse of a cell?",
    },

    # Q4: unintended skills
    {
        "id": "Q4_1",
        "type": "unintended_skill",
        "input": "What is 2 + 2? Pick an answer from the following options:\n A.
↪1 \n B.2 \n C.3 \n D.4 \n E.5 "
    },
    {
        "id": "Q4_2",
        "type": "unintended_skill",
        "input": "Explain what Parallelism means mathematically, you have a 30_
↪word limit."
    },
]

results = []

for ex in examples:
    user_prompt = ex["input"]

    base_out = run_chat(baseline_model, baseline_tokenizer, user_prompt)
    tuned_out = run_chat(model, tokenizer, user_prompt)

    results.append({
        "id": ex["id"],
        "type": ex["type"],
        "input": user_prompt,
        "base_output": base_out,
        "tuned_output": tuned_out,
    })

for r in results:
    print("=" * 80)
    print(f"Example {r['id']} ({r['type']})")
    print("- Input:")
    print(r["input"])
    print("\n- Base model output:\n", r["base_output"])
    print("\n- Finetuned/RAG model output:\n", r["tuned_output"])
    print()

```

Baseline Model loaded.

=====

Example Q3_1 (catastrophic_forgetting)

- Input:

If it rains, I bring an umbrella out. So if I do not bring an umbrella out, it must mean it is not raining. Is this true?

- Base model output:

Your statement that "if it rains, I bring an umbrella out" and "if I do not bring an umbrella out, it must mean it is not raining" can be analyzed in terms of logical reasoning.

Let's break down the statements:

1. **Statement 1:** If it rains, then I bring an umbrella out.
2. **Statement 2:** If I do not bring an umbrella out, then it is not raining.

To determine whether these statements are logically equivalent or contradictory, we need to consider their implications.

Analysis:

- **Statement 1** states: "If it rains, then I bring an umbrella

- Finetuned/RAG model output:

Yes, your statement is correct. If you do not bring an umbrella out when it is raining, then it must be raining because rain is the only thing that would cause someone to need an umbrella.

=====

Example Q3_2 (catastrophic_forgetting)

- Input:

What is the powerhouse of a cell?

- Base model output:

The powerhouse of a cell is the mitochondria. Mitochondria are small, double-membrane-bound organelles found in all eukaryotic cells, including plant and animal cells. They play a crucial role in cellular respiration, which is the process by which cells convert nutrients into energy.

Key functions of mitochondria include:

1. **Energy Production:** Mitochondria are responsible for producing ATP (adenosine triphosphate), the primary energy currency of cells. This occurs through a series of complex biochemical reactions that generate high-energy electrons.
2. **Respiratory Function:** Mitochondria are involved in the aerobic

- Finetuned/RAG model output:

The powerhouse of a cell is typically referred to as the nucleus. The nucleus contains the cell's genetic material (DNA) and is responsible for controlling various cellular processes such as DNA replication, transcription, and translation.

=====

Example Q4_1 (unintended_skill)

- Input:

What is 2 + 2? Pick an answer from the following options:

- A.1
- B.2
- C.3
- D.4
- E.5

- Base model output:

The correct answer is B. 2.

When you add 2 to itself, you get 2 again:

$2 + 2 = 4$

So, the final answer is B. 2.

- Finetuned/RAG model output:

B.

=====

Example Q4_2 (unintended_skill)

- Input:

Explain what Parallelism means mathematically, you have a 30 word limit.

- Base model output:

Parallelism in mathematics refers to the ability of an algorithm or system to execute multiple tasks simultaneously without any delay or race conditions. This concept is crucial for optimizing performance and ensuring that all computations can be completed efficiently when resources are limited. In parallel computing, tasks are divided into smaller subtasks (or "threads") that run concurrently on different processors or cores, allowing them to work independently until their completion. This approach significantly reduces overall execution time compared to sequential processing, making it more efficient for large-scale applications.

- Finetuned/RAG model output:

Parallelism in mathematics refers to the ability of a system or algorithm to distribute tasks across multiple processors or cores simultaneously. This allows for faster computation and better utilization of computational resources.

[45] :